

Automated FIR Filter Synthesis

Genetic Algorithms for Real-Valued and CSD Designs

Simone Deiana

February 16, 2026

- 1 Introduction and Baseline
- 2 Hardware Constraints & CSD
- 3 CSD-Compatible GA (Literature)
- 4 Enhanced CSD GA
- 5 Experimental Results
- 6 Conclusion

- A digital Finite Impulse Response (FIR) filter is characterized by the transfer function:

$$H(z) = \sum_{n=0}^N a_n z^{-n}$$

- **Goal:** Find coefficients a_n (order N) to match a desired frequency response.
- **Evolutionary Approach:** An individual is a vector in $\mathbb{R}^N$.

Baseline: Real-Valued GA - Implementation

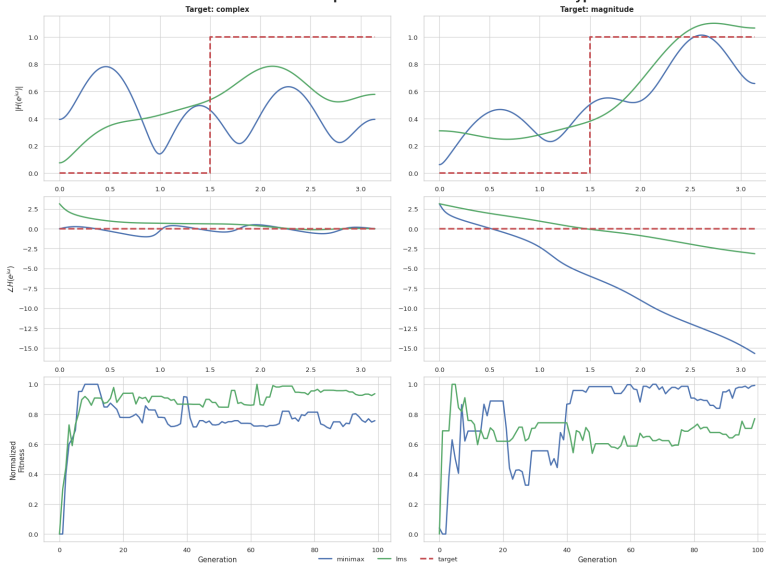
Concept: A simple proof of concept using infinite-precision coefficients.

GA Structure:

- **Chromosome:** $x = (x_1, \dots, x_n)^T$, where $x \in \mathbb{R}^n$.
- **Init:** Uniform sampling $[-1, 1]^n$.
- **Mutation:** Gaussian (non-adaptive).
- **Crossover:** Intermediate recombination.
- **Selection:** Roulette wheel.
- **Fitness:** Minimax (L_∞) or LMS (L_2) error for complex or magnitude target.

Baseline: Real-Valued GA - Test run

GA FIR Evolution — Comparison Across Modes and Fitness Types



Baseline: Real-Valued GA - Issues

Problem: While mathematically sound, infinite precision is not feasible for hardware implementation due to cost and complexity.

Solution: Explore a solution space that natively represents coefficients in a hardware-friendly way (*binary strings?*).

The Hardware Challenge

The Cost of Multipliers

In hardware, multipliers are expensive. Hardware costs increase with the filter order and coefficient precision. We need more than a simple binary representation.

Solution: Canonical Signed Digit (CSD)

- A Radix-2 signed-digit system: $d_k \in \{-1, 0, +1\}$.
- **Key Constraint:** No two adjacent digits may be non-zero.

$$d_k \neq 0 \Rightarrow d_{k-1} = d_{k+1} = 0$$

- **Benefit:** Multiplication becomes simple bit shifts and adds/subs.

$$h[k] \cdot x[n - k] = \sum_i d_i (x[n - k] \ll i)$$

The Hardware Challenge

Canonical Signed Digit (CSD):

$$\begin{aligned} \text{coefficient } 0.421875 &\Rightarrow 2^{-1} - 2^{-4} - 2^{-6} \\ &\Rightarrow \boxed{0 \ 1 \ 0 \ 0 \ \bar{1} \ 0 \ \bar{1} \ 0} \end{aligned}$$

LSB

Method 1: Basic CSD GA (The Paper)

Based on work by A. Lee, M. Ahmadi, et al.

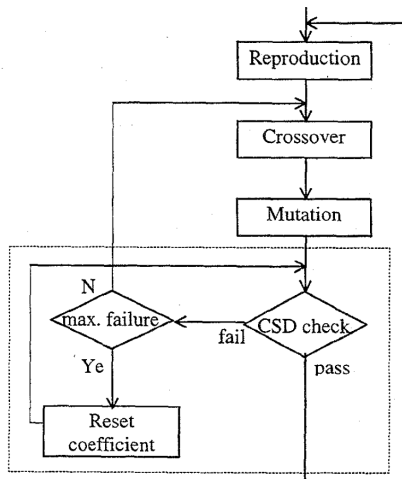
GA Implementation:

- **Encoding:** Coefficients are strings of ternary digits corresponding to powers-of-two.
- **Initialization:** One non-zero bit per coefficient ($h = 2^{-k}$).
- **Mutation:** Random choice from $\{-1, 0, 1\} \setminus \{symbol\}$.
- **Crossover:** Two-point crossover on the genome.

Design constraints:

- **Wordlength:** Number of bits our hardware can handle.
- **N_{digits}:** Maximum number of nonzero digits in a coefficient (*more nonzero digits = more circuit complexity*).

Method 1: Basic CSD GA (The Paper)



The Compliance Issue

Standard operators often break CSD constraints (adjacent non-zeros) or exceed the max digit limit (N_{digits}).

Fix: The algorithm retries operators up to N_{max} times. If it fails, the broken coefficients are reset.

Method 1: Basic CSD GA - Test run



Method 1: Basic CSD GA - Issues

Problem 1: Mutation and crossover are “dumb” and simple to implement, but the risk of breaking both the CSD and N_{digits} constraints is very high, leading to a lot of failed checks and coefficient resets.

Problem 2: The paper uses roulette wheel selection, which can be improved upon. Elitism was not included.

Solution: Implement a “CSD-friendly” mutation and crossover, and use more modern strategies for the GA.

Method 2: Enhanced CSD GA (Our Approach)

We improved the algorithm to preserve CSD compliance and convergence speed.

1. CSD-Preserving Mutation

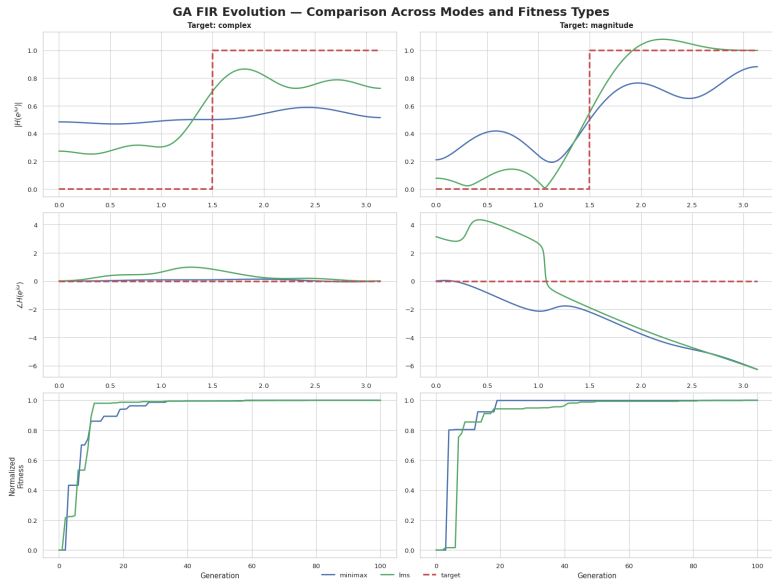
- Aggressive strategy.
- If mutation creates adjacent non-zeros, adjacent digits are forced to 0.
- Prevents invalid states without discarding the operation.

2. Coefficient-wise Crossover

- Instead of cutting *inside* a coefficient (which breaks structure), we crossover entire coefficients.
- Guarantees offspring are valid if parents were valid.

3. Selection: Switched to Tournament Selection + Elitism.

Method 2: Enhanced CSD GA - Test run



Experimental Setup

Parameters:

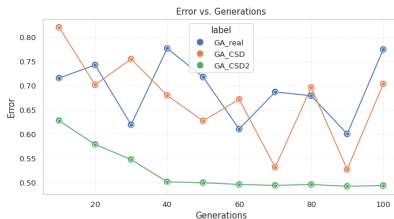
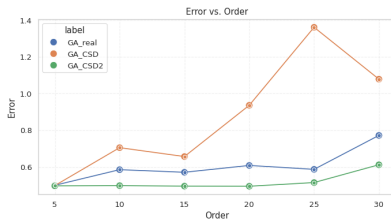
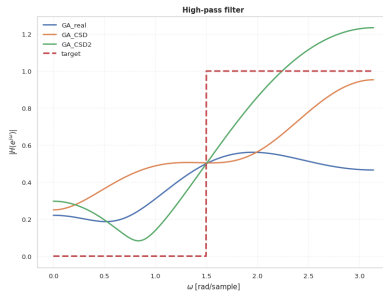
- Population: 100
- Wordlength: 8
- Max Non-zero Digits (N_{digits}): 3

Target Filters:

- 1 Ideal High-pass
- 2 Ideal Band-pass
- 3 Differentiator ($H(j\omega) = j\omega$)
- 4 Gaussian Low-pass

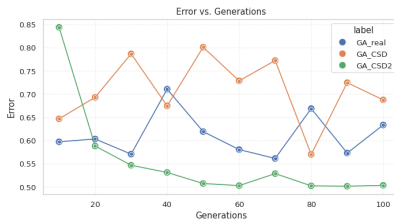
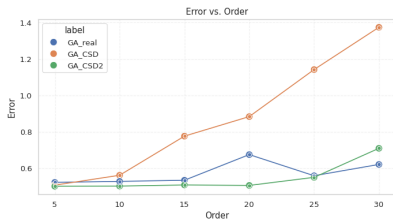
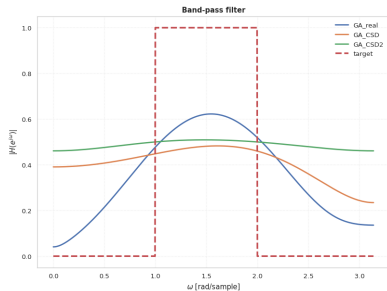
Results: High-pass filter

$$H(\omega) = \begin{cases} 0, & |\omega| < 1.5, \\ 1, & 1.5 \leq |\omega| \leq \pi, \end{cases}$$



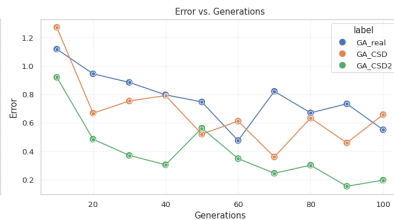
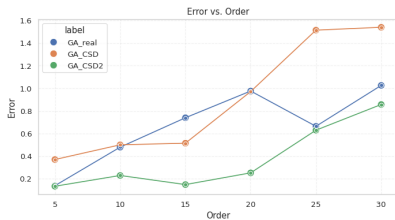
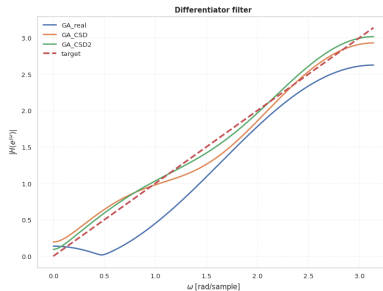
Results: Band-pass filter

$$H(\omega) = \begin{cases} 0, & |\omega| < 1, \\ 1, & 1 \leq |\omega| \leq 2, \\ 0, & |\omega| > 2, \end{cases}$$



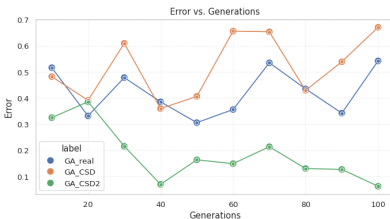
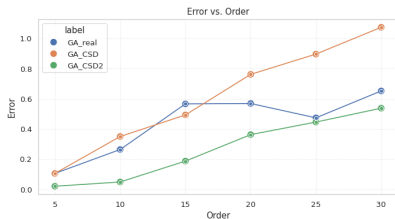
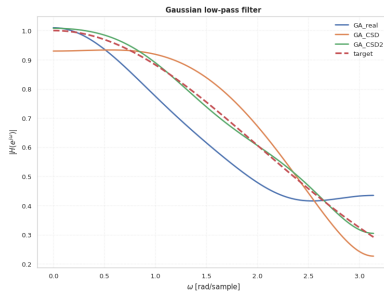
Results: Differentiator filter

$$H(\omega) = j\omega$$



Results: Gaussian low-pass filter

$$H(\omega) = e^{-(\omega/2)^2}$$



Results: Convergence & Error

- **Real-Valued:** Baseline, fastest.
- **Basic CSD (Paper):**
 - High error rates when filter order increases, could be because of frequent coefficient resets hurting progress.
 - 200% slower than real valued.
 - Consistently worst performance.
- **Enhanced CSD:**
 - Best performance across the board.
 - Slightly faster than base CSD.
 - Consistent results thanks to elitism.

The Impact of Elitism

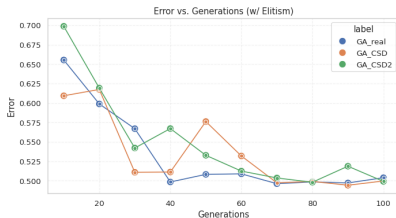
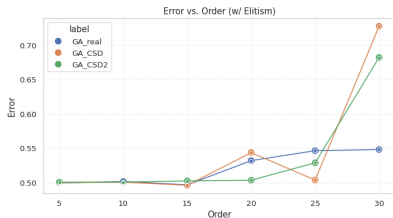
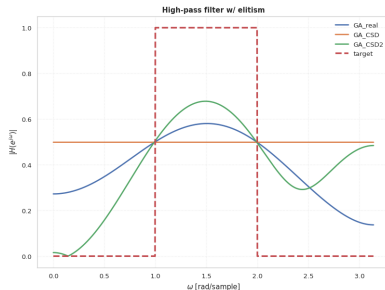
Observation

Without elitism, both the real baseline and the basic CSD GAs struggled to maintain good solutions and displayed similar performance.

Question: What if we enable elitism for all implementations?

Results: High-pass filter w/ elitism

$$H(\omega) = \begin{cases} 0, & |\omega| < 1.5, \\ 1, & 1.5 \leq |\omega| \leq \pi, \end{cases}$$



The Impact of Elitism - Results

- Elitism constitutes a huge boost in performance.
- The competing techniques become much more comparable and consistent.
- Ternary encoding still has some problems converging at a high number of coefficients.

Conclusion

- 1 **Feasibility:** It is possible to automatically synthesize hardware-ready FIR filters using Genetic Algorithms.
- 2 **Encoding:** Direct CSD encoding is superior to real-valued optimization for hardware targets, but requires careful handling of constraints.
- 3 **Operator Design:** The "Enhanced" approach (CSD-preserving operators + Coefficient-wise crossover) outperforms the "Retry" method proposed in the literature.
- 4 **Key Insight:** Elitism is not optional; it is essential for convergence in highly constrained search spaces like CSD.